AI-Based Network Pathfinding & Attack Simulation System

Course: Artificial Intelligence
Institute: COMSATS University Islamabad, CUI Abbottabad
Due Date: April 30, 2026

Project Overview

This notebook simulates an attacker traversing a computer network modelled as a weighted graph.
Seven AI search algorithms are implemented, compared, and visualised:

- **Uninformed:** BFS, DFS
- **Informed:** UCS, A*
- **Local:** Hill Climbing
- **Adversarial:** Minimax, Alpha-Beta Pruning

1. Install & Import Dependencies

```
# All libraries used are part of Python standard library — no pip installs needed
import heapq, time, math, random
from collections import deque
print(" Libraries imported successfully")
```

```
 Libraries imported successfully
```

2. Network Graph Definition

Nodes represent network entities; edges represent connections with vulnerability weights.

```
# ── Node metadata: name → (type, canvas_x, canvas_y) ──
NODE_INFO = {
    "Internet":    ("Entry",    100, 300),
    "Firewall":    ("Firewall", 220, 300),
    "WebServer":   ("Server",   340, 180),
```

```
    "AppServer":   ("Server",   340, 300),
    "Workstation1":("Client",   340, 420),
    "Workstation2":("Client",   460, 480),
    "InternalFW":  ("Firewall", 460, 300),
    "DBServer":    ("Database", 580, 200),
    "AdminPC":     ("Client",   580, 360),
    "CoreDB":      ("Database", 700, 280),
}

# ── Weighted adjacency list: node → [(neighbor, cost)] ──
GRAPH = {
    "Internet":    [("Firewall", 2)],
    "Firewall":    [("Internet", 2), ("WebServer", 3), ("AppServer", 5), ("Work
    "WebServer":   [("Firewall", 3), ("AppServer", 2), ("InternalFW", 6)],
    "AppServer":   [("Firewall", 5), ("WebServer", 2), ("InternalFW", 3), ("Wor
    "Workstation1":[("Firewall", 7), ("AppServer", 4), ("Workstation2", 2)],
    "Workstation2":[("Workstation1", 2), ("AdminPC", 5), ("InternalFW", 4)],
    "InternalFW":  [("WebServer", 6), ("AppServer", 3), ("Workstation2", 4), ('
    "DBServer":    [("InternalFW", 4), ("CoreDB", 2), ("AdminPC", 3)],
    "AdminPC":     [("InternalFW", 3), ("DBServer", 3), ("Workstation2", 5), ('
    "CoreDB":      [("DBServer", 2), ("AdminPC", 4)],
}

START = "Internet"
GOAL  = "CoreDB"

print(f" Graph loaded: {len(GRAPH)} nodes, Start={START}, Goal={GOAL}")
```

```
 Graph loaded: 10 nodes, Start=Internet, Goal=CoreDB
```

3. Helper Utilities

```
def path_cost(path, graph):
    cost = 0
    for i in range(len(path) - 1):
        for nb, w in graph.get(path[i], []):
            if nb == path[i+1]:
                cost += w; break
    return cost
print(" Helpers ready")
```

```
 Helpers ready
```

4. Uninformed Search: BFS & DFS

```
def bfs(graph, start, goal):
    t0 = time.perf_counter()
    queue = deque([[start]])
    visited = {start}
    nodes_expanded = 0
    while queue:
        path = queue.popleft()
        node = path[-1]
        nodes_expanded += 1
        if node == goal:
            return path, path_cost(path, graph), nodes_expanded, (time.perf_cou
        for neighbor, _ in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return [], 0, nodes_expanded, (time.perf_counter()-t0)*1000

def dfs(graph, start, goal):
    t0 = time.perf_counter()
    stack = [[start]]
    visited = set()
    nodes_expanded = 0
    while stack:
        path = stack.pop()
        node = path[-1]
        if node in visited: continue
        visited.add(node); nodes_expanded += 1
        if node == goal:
            return path, path_cost(path, graph), nodes_expanded, (time.perf_cou
        for neighbor, _ in reversed(graph.get(node, [])):
            if neighbor not in visited:
                stack.append(path + [neighbor])
    return [], 0, nodes_expanded, (time.perf_counter()-t0)*1000

# Test
print("BFS:", bfs(GRAPH, START, GOAL)[:3])
print("DFS:", dfs(GRAPH, START, GOAL)[:3])
```

```
BFS: (['Internet', 'Firewall', 'WebServer', 'InternalFW', 'DBServer', 'CoreDB'],
DFS: (['Internet', 'Firewall', 'WebServer', 'AppServer', 'InternalFW', 'Workstat
```

5. Informed Search: UCS & A*

The **heuristic h(n)** for A* is the scaled Euclidean distance between node positions — representing an estimated topological closeness to the target.

```
def ucs(graph, start, goal):
    t0 = time.perf_counter()
    heap = [(0, start, [start])]
    visited = {}
    nodes_expanded = 0
    while heap:
        cost, node, path = heapq.heappop(heap)
        if node in visited: continue
        visited[node] = cost; nodes_expanded += 1
        if node == goal:
            return path, cost, nodes_expanded, (time.perf_counter()-t0)*1000
        for neighbor, w in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(heap, (cost+w, neighbor, path+[neighbor]))
    return [], 0, nodes_expanded, (time.perf_counter()-t0)*1000

def heuristic(node, goal, node_info):
    x1,y1 = node_info[node][1], node_info[node][2]
    x2,y2 = node_info[goal][1],  node_info[goal][2]
    return math.sqrt((x2-x1)**2 + (y2-y1)**2) / 80

def astar(graph, start, goal, node_info):
    t0 = time.perf_counter()
    h = lambda n: heuristic(n, goal, node_info)
    heap = [(h(start), 0, start, [start])]
    g_score = {start: 0}
    nodes_expanded = 0
    while heap:
        f, g, node, path = heapq.heappop(heap)
        nodes_expanded += 1
        if node == goal:
            return path, g, nodes_expanded, (time.perf_counter()-t0)*1000
        for neighbor, w in graph.get(node, []):
            tg = g + w
            if tg < g_score.get(neighbor, float('inf')):
                g_score[neighbor] = tg
                heapq.heappush(heap, (tg+h(neighbor), tg, neighbor, path+[neigh
    return [], 0, nodes_expanded, (time.perf_counter()-t0)*1000

print("UCS:", ucs(GRAPH, START, GOAL)[:3])
print("A*: ", astar(GRAPH, START, GOAL, NODE_INFO)[:3])
```

```
UCS: (['Internet', 'Firewall', 'AppServer', 'InternalFW', 'DBServer', 'CoreDB'],
A*:  (['Internet', 'Firewall', 'AppServer', 'InternalFW', 'DBServer', 'CoreDB'],
```

6. Local Search: Hill Climbing

> **Local Maximum Demo:** Hill Climbing can get stuck when no neighbor improves the heuristic, even if the goal is reachable via a longer detour.

```
def hill_climbing(graph, start, goal, node_info):
    t0 = time.perf_counter()
    current = start
    path = [current]
    visited = {current}
    nodes_expanded = 0
    h = lambda n: heuristic(n, goal, node_info)

    while current != goal:
        neighbors = [(h(nb), nb) for nb, _ in graph.get(current, []) if nb not
        nodes_expanded += 1
        if not neighbors:
            path.append(f"[STUCK@{current}]"); break
        neighbors.sort()
        best_h, best_nb = neighbors[0]
        if best_h >= h(current):
            path.append(f"[STUCK@{current}]"); break
        current = best_nb
        path.append(current); visited.add(current)

    elapsed = (time.perf_counter()-t0)*1000
    clean = [n for n in path if not n.startswith("[")]
    return path, path_cost(clean, graph), nodes_expanded, elapsed

result = hill_climbing(GRAPH, START, GOAL, NODE_INFO)
print("Hill Climbing path:", result[0])
print("Stuck?" , any(str(p).startswith("[STUCK") for p in result[0]))
```

```
Hill Climbing path: ['Internet', 'Firewall', 'AppServer', 'InternalFW', 'AdminPC
Stuck? False
```

7. Adversarial Search: Minimax & Alpha-Beta Pruning

- **Attacker (Maximizer):** Tries to reach CoreDB
- **Defender (Minimizer):** Tries to block the path
- **Alpha-Beta Pruning** skips branches that cannot affect the outcome, making it faster than plain Minimax.

```
def minimax(graph, node, depth, is_max, goal, node_info, visited=None):
    if visited is None: visited = set()
    h = lambda n: -heuristic(n, goal, node_info)
    if node == goal or depth == 0 or node in visited:
```

```
        return h(node), node
    visited = visited | {node}
    neighbors = [nb for nb,_ in graph.get(node,[])]
    if not neighbors: return h(node), node
    if is_max:
        best, bn = float('-inf'), None
        for nb in neighbors:
            v,_ = minimax(graph,nb,depth-1,False,goal,node_info,visited)
            if v>best: best,bn=v,nb
        return best,bn
    else:
        best, bn = float('inf'), None
        for nb in neighbors:
            v,_ = minimax(graph,nb,depth-1,True,goal,node_info,visited)
            if v<best: best,bn=v,nb
        return best,bn

def alphabeta(graph, node, depth, alpha, beta, is_max, goal, node_info, visited=
    if visited is None: visited = set()
    h = lambda n: -heuristic(n, goal, node_info)
    if node == goal or depth == 0 or node in visited:
        return h(node), node
    visited = visited | {node}
    neighbors = [nb for nb,_ in graph.get(node,[])]
    if not neighbors: return h(node), node
    if is_max:
        best, bn = float('-inf'), None
        for nb in neighbors:
            v,_ = alphabeta(graph,nb,depth-1,alpha,beta,False,goal,node_info,vis
            if v>best: best,bn=v,nb
            alpha = max(alpha,best)
            if beta<=alpha: break
        return best,bn
    else:
        best, bn = float('inf'), None
        for nb in neighbors:
            v,_ = alphabeta(graph,nb,depth-1,alpha,beta,True,goal,node_info,visi
            if v<best: best,bn=v,nb
            beta = min(beta,best)
            if beta<=alpha: break
        return best,bn

def run_minimax_path(graph, start, goal, node_info, use_alphabeta=False):
    t0 = time.perf_counter()
    path=[start]; visited=set(); nodes_expanded=0; current=start; DEPTH=4
    while current!=goal and len(path)<25:
        visited.add(current); nodes_expanded+=1
        neighbors=[nb for nb,_ in graph.get(current,[]) if nb not in visited]
        if not neighbors: break
        if use_alphabeta:
            _,best=alphabeta(graph,current,DEPTH,float('-inf'),float('inf'),True
        else:
```

```
            _,best=minimax(graph,current,DEPTH,True,goal,node_info,visited.copy(
        if best is None or best in visited:
            best=min(neighbors, key=lambda n: heuristic(n,goal,node_info))
        path.append(best); current=best
    elapsed=(time.perf_counter()-t0)*1000
    return path, path_cost(path,graph), nodes_expanded, elapsed

print("Minimax   :", run_minimax_path(GRAPH,START,GOAL,NODE_INFO,False)[:2])
print("Alpha-Beta:", run_minimax_path(GRAPH,START,GOAL,NODE_INFO,True)[:2])
```

```
Minimax   : (['Internet', 'Firewall', 'AppServer', 'InternalFW', 'DBServer', 'Cc
Alpha-Beta: (['Internet', 'Firewall', 'AppServer', 'InternalFW', 'DBServer', 'Cc
```

8. Comparative Analysis Table

```
results = {
    "BFS":          bfs(GRAPH, START, GOAL),
    "DFS":          dfs(GRAPH, START, GOAL),
    "UCS":          ucs(GRAPH, START, GOAL),
    "A*":           astar(GRAPH, START, GOAL, NODE_INFO),
    "Hill Climbing":hill_climbing(GRAPH, START, GOAL, NODE_INFO),
    "Minimax":      run_minimax_path(GRAPH, START, GOAL, NODE_INFO, False),
    "Alpha-Beta":   run_minimax_path(GRAPH, START, GOAL, NODE_INFO, True),
}

print(f"\n{'='*100}")
print(f"{'Algorithm':<16} {'Discovered Path':<50} {'Cost':>6} {'Nodes':>7} {'Tim
print(f"{'='*100}")
for algo, (path, cost, exp, ms) in results.items():
    clean_path = [p for p in path if not str(p).startswith("[")]
    ps = " → ".join(clean_path)
    if len(ps)>48: ps=ps[:45]+"..."
    print(f"{algo:<16} {ps:<50} {cost:>6.1f} {exp:>7} {ms:>10.3f}")
print(f"{'='*100}")
```

```
====================================================================================
Algorithm        Discovered Path                                    Cost   Nod
====================================================================================
BFS              Internet → Firewall → WebServer → InternalFW ...     17.0
DFS              Internet → Firewall → WebServer → AppServer →...     24.0
UCS              Internet → Firewall → AppServer → InternalFW ...     16.0
A*               Internet → Firewall → AppServer → InternalFW ...     16.0
Hill Climbing    Internet → Firewall → AppServer → InternalFW ...     17.0
Minimax          Internet → Firewall → AppServer → InternalFW ...     16.0
Alpha-Beta       Internet → Firewall → AppServer → InternalFW ...     16.0
====================================================================================
```

9. Network Visualisation (matplotlib)

The graph below shows all nodes, edges, and highlights the A* optimal path.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

NODE_COLORS_MPL = {
    "Entry":"#e74c3c","Firewall":"#e67e22",
    "Server":"#3498db","Client":"#2ecc71","Database":"#9b59b6"
}

def draw_network(highlight_path=None, title="Network Topology"):
    fig, ax = plt.subplots(figsize=(13, 7))
    ax.set_facecolor("#0f3460"); fig.patch.set_facecolor("#1a1a2e")
    ax.set_title(title, color="white", fontsize=14, fontweight="bold")
    ax.axis("off")

    hp_edges = set()
    if highlight_path:
        valid = [n for n in highlight_path if not str(n).startswith("[")]
        hp_edges = set(zip(valid, valid[1:]))

    for node, neighbors in GRAPH.items():
        x1,y1 = NODE_INFO[node][1], NODE_INFO[node][2]
        for nb,w in neighbors:
            x2,y2 = NODE_INFO[nb][1], NODE_INFO[nb][2]
            is_hl = (node,nb) in hp_edges or (nb,node) in hp_edges
            ax.plot([x1,x2],[y1,y2], color="#e94560" if is_hl else "#334155",
                    lw=3 if is_hl else 1, zorder=1)
            ax.text((x1+x2)/2, (y1+y2)/2+6, str(w),
                    ha="center", va="center", color="#94a3b8", fontsize=7)

    for node,(ntype,x,y) in NODE_INFO.items():
        color = NODE_COLORS_MPL.get(ntype,"#888")
        ec = "#e94560" if node==START else ("#f0d000" if node==GOAL else
              ("#ffffff" if highlight_path and node in highlight_path else "#33
        lw = 3 if node in (START,GOAL) or (highlight_path and node in highlight
        circle = plt.Circle((x,y), 28, color=color, ec=ec, lw=lw, zorder=2)
        ax.add_patch(circle)
        ax.text(x, y, node[:9], ha="center", va="center",
                color="white", fontsize=7, fontweight="bold", zorder=3)

    patches = [mpatches.Patch(color=c, label=t) for t,c in NODE_COLORS_MPL.item
    ax.legend(handles=patches, loc="lower left", facecolor="#16213e",
              labelcolor="white", fontsize=8)
    ax.set_xlim(50,760); ax.set_ylim(100,550)
```

```
    plt.tight_layout(); plt.show()

astar_path = results["A*"][0]
draw_network(highlight_path=astar_path, title=f"A* Optimal Attack Path: {' → '.
```